

Glosario - Semana 07

Repository Pattern y Unit of Work

A

Abstract Repository

Interfaz o clase base que define el contrato que deben cumplir los repositorios concretos. Permite intercambiar implementaciones (DB real, fake, etc.).

Aggregate

En Domain-Driven Design, un cluster de objetos de dominio que se tratan como una unidad. El Repository típicamente opera sobre aggregates.

B

BaseRepository

Clase genérica que implementa operaciones CRUD comunes. Usa `Generic[T]` para trabajar con cualquier tipo de entidad.

```
class BaseRepository(Generic[T]):
    def get_by_id(self, id: int) -> T | None: ...
    def add(self, entity: T) -> T: ...
```

Bounded Context

Límite conceptual donde un modelo de dominio particular es definido y aplicable. Los repositorios pertenecen a un bounded context específico.

C

CRUD

Create, Read, Update, Delete - Las cuatro operaciones básicas de persistencia que un repositorio típicamente implementa.

Commit

Operación que confirma todos los cambios pendientes en una transacción de base de datos, haciéndolos permanentes.

```
session.commit() # Guarda cambios permanentemente
```

D

Data Access Layer (DAL)

Capa de la aplicación responsable de la comunicación con la base de datos. Los repositorios forman parte de esta capa.

Dependency Injection (DI)

Patrón donde las dependencias se pasan a un objeto en lugar de que el objeto las cree. Los services reciben repositorios por inyección.

```
class UserService:
    def __init__(self, repo: UserRepository):
        self.repo = repo # Inyectado, no creado
```

F

Fake Repository

Implementación de repositorio que usa estructuras en memoria (dict, list) en lugar de base de datos real. Útil para testing.

```
class FakeUserRepository:
    def __init__(self):
        self._data: dict[int, User] = {}
```

Flush

Operación que sincroniza los cambios pendientes con la base de datos sin hacer commit. Permite obtener IDs generados.

```
session.add(entity)
session.flush() # Sincroniza, obtiene ID
# entity.id ahora tiene valor
session.commit() # Confirma transacción
```

G

Generic

En Python, clase que puede trabajar con diferentes tipos mediante parámetros de tipo. Se define usando `Generic[T]` .

```
from typing import TypeVar, Generic

T = TypeVar("T")

class Repository(Generic[T]):
    def get(self, id: int) -> T: ...
```

I

Identity Map

Patrón que asegura que cada objeto se carga solo una vez por sesión. SQLAlchemy Session implementa este patrón automáticamente.

Inversion of Control (IoC)

Principio donde el control del flujo se invierte: en lugar de que el código llame a bibliotecas, el framework llama al código del usuario.

L

Lazy Loading

Técnica donde los datos relacionados se cargan solo cuando se acceden, no al cargar el objeto principal.

M

Mock

Objeto que simula el comportamiento de objetos reales de manera controlada. Diferente de Fake: los mocks verifican interacciones.

```
from unittest.mock import Mock

mock_repo = Mock()
mock_repo.get_by_id.return_value = user
```

P

Persistence Ignorance

Principio donde las entidades de dominio no conocen cómo se persisten. El repositorio encapsula esa lógica.

Port

En arquitectura hexagonal, interfaz que define cómo el dominio se comunica con el exterior. Un repositorio es un "port" de salida.

R

Repository

Patrón que encapsula la lógica de acceso a datos, proporcionando una interfaz de colección para acceder a objetos del dominio.

```
class UserRepository:
    def get_by_id(self, id: int) -> User | None: ...
    def get_by_email(self, email: str) -> User | None: ...
    def add(self, user: User) -> User: ...
```

Rollback

Operación que revierte todos los cambios pendientes en una transacción, restaurando el estado anterior.

```
try:
    # operaciones...
    session.commit()
except:
    session.rollback() # Revierte cambios
```

S

Session

En SQLAlchemy, objeto que gestiona las operaciones de persistencia. Implementa Identity Map y Unit of Work.

Service Layer

Capa que contiene la lógica de negocio de la aplicación. Los services usan repositorios para acceder a datos.

```
class UserService:
    def __init__(self, uow: UnitOfWork):
        self.uow = uow

    def create_user(self, data: UserCreate) -> User:
        # Lógica de negocio aquí
        return self.uow.users.add(user)
```

T

Transaction

Secuencia de operaciones de base de datos que se ejecutan como una unidad atómica. O todas se completan o ninguna.

TypeVar

En Python, variable de tipo que representa un tipo genérico. Se usa con `Generic` para crear clases genéricas.

```
from typing import TypeVar

T = TypeVar("T", bound=Base) # T debe heredar de Base
```

U

Unit of Work (UoW)

Patrón que mantiene una lista de objetos afectados por una transacción y coordina la escritura de cambios.

```
class UnitOfWork:
    def __enter__(self):
        self.users = UserRepository(self.session)
        self.tasks = TaskRepository(self.session)
        return self

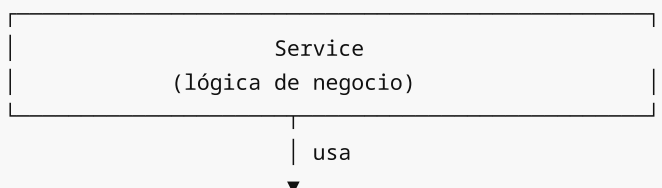
    def commit(self):
        self.session.commit()

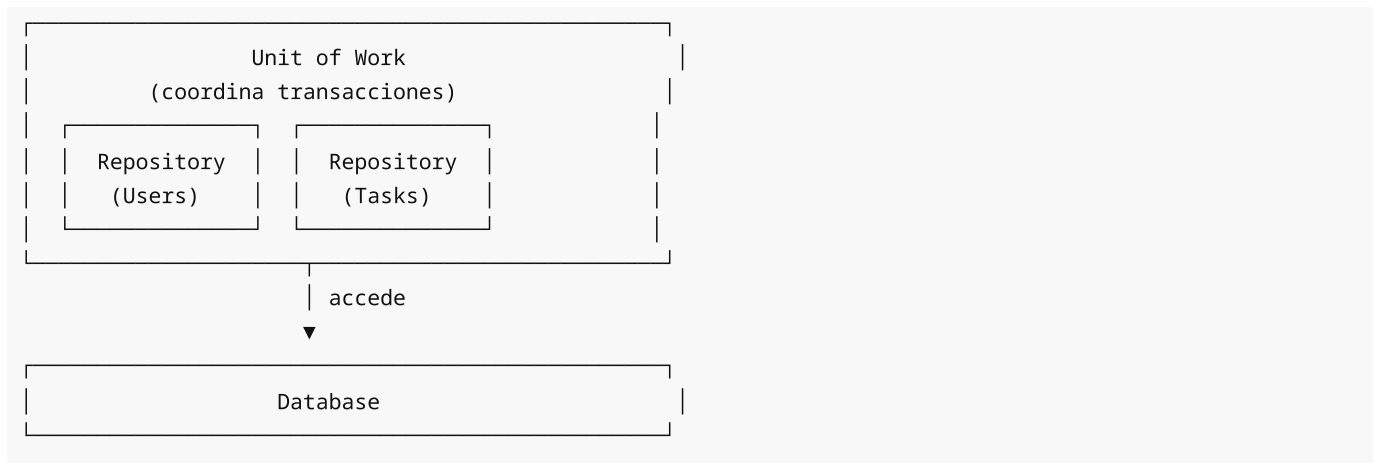
    def rollback(self):
        self.session.rollback()
```

Unit Test

Test que verifica una unidad aislada de código (función, clase). Los fake repositories permiten tests unitarios de services.

Diagrama de Relaciones





Referencias

- Fowler, M. - Patterns of Enterprise Application Architecture
- Evans, E. - Domain-Driven Design
- Percival, H. & Gregory, B. - Architecture Patterns with Python